

What is Pandas?

Pandas is an open-source Python library used for data analysis and data manipulation. It helps in reading, cleaning, organizing, analyzing, and storing data efficiently.

Developed by: Wes McKinney (2008)

Features of Pandas Fast and easy data analysis Handles large datasets Reads data from multiple file formats Cleans missing data Filters and sorts data Groups and summarizes data Merges multiple datasets Time series analysis Works well with NumPy and Matplotlib

Installation

```
pip install pandas
```

Import Pandas

```
import pandas as pd
```

Check version

```
print(pd.__version__)
```

Pandas Data Structures

1. Series (1D)

A Series is a one-dimensional labeled array

Example

```
import pandas as pd

s = pd.Series([10,20,30,40])
print(s)
```

Output

```
0 10 1 20 2 30 3 40 dtype: int64
```

2. DataFrame (2D)

DataFrame is a two-dimensional table with rows and columns

Example

```
data = {
    "Name": ["Rahul", "Priya", "Amit"],
    "Age": [21, 22, 20]
}

df = pd.DataFrame(data)

print(df)
```

Output

Name	Age
------	-----

O Rahul 21 1 Priya 22 2 Amit 20

Creating DataFrame

```
data = {  
    "Name": ["A", "B", "C"],  
    "Marks": [80, 90, 85]  
}  
  
df = pd.DataFrame(data)
```

From List

```
data = [[1, "Ram"], [2, "Shyam"]]  
  
df = pd.DataFrame(data,  
    columns=["ID", "Name"])
```

From NumPy Array

```
import numpy as np  
  
arr=np.array([[1,2],[3,4]])  
  
df=pd.DataFrame(arr)
```

Reading Files

CSV

```
df=pd.read_csv("student.csv")
```

Excel

```
df=pd.read_excel("student.xlsx")
```

JSON

```
df=pd.read_json("data.json")
```

Saving Files

CSV

```
df.to_csv("output.csv")
```

```
df.to_csv("output.csv")
```

Excel

```
df.to_excel("output.xlsx")
```

Viewing Data

First rows

```
df.head()
```

Last rows

```
df.tail()
```

Information

```
df.info()
```

Summary

```
df.describe()
```

Shape

```
df.shape
```

Columns

```
df.columns
```

Index

```
df.index
```

Selecting Data Column

```
df["Name"]
```

Multiple columns

```
Multiple columns
```

```
df[["Name", "Age"]]
```

Single row

```
df.iloc[0]
```

Specific row and column

```
df.iloc[0,1]
```

Label-based

```
df.loc[0]
```

Filtering Data

```
df[df["Age"]>20]
```

Multiple conditions

```
df[(df["Age"]>20)&(df["Marks"]>80)]
```

Adding Columns

```
df["City"]="Kolhapur"
```

New column

```
df["Total"]=df["Marks"]+5
```

Updating Data

```
df["Age"]=25
```

Update one value

```
df.loc[0,"Age"]=30
```

Deleting Data

Delete column

```
df.drop("Age",axis=1)
```

```
df.drop("Age",axis=1)
```

Delete row

```
df.drop(0)
```

Sorting

Ascending

```
df.sort_values("Marks")
```

Descending

```
df.sort_values("Marks",ascending=False)
```

Missing Values

Check

```
df.isnull()
```

Count

```
df.isnull().sum()
```

Remove

```
df.dropna()
```

Fill

```
df.fillna(0)
```

Duplicate Data

Find

Duplicate Data

Find

Remove

```
df.drop_duplicates()
```

Rename Columns

```
df.rename(columns={"Age": "Student_Age"})
```

Statistical Functions

```
df.mean()
```

```
df.sum()
```

```
df.min()
```

```
df.max()
```

```
df.count()
```

```
df.std()
```

```
df.var()
```

```
df.median()
```

Group By

```
df.groupby("Department").sum()
```

Average

```
df.groupby("Department").mean()
```

Merge DataFrames

```
pd.merge(df1, df2, on="ID")
```

Concatenate

```
pd.concat([df1, df2])
```

Join

```
df1.join(df2)
```

Pivot Table

```
pd.pivot_table(df,  
values="Marks",  
index="Department")
```

Apply Function

```
df["Marks"] = df["Marks"].apply(lambda x: x+5)
```

String Functions

```
df["Name"].str.upper()

df["Name"].str.lower()

df["Name"].str.len()

df["Name"].str.replace("A", "X")
```

Date Functions

```
pd.to_datetime(df["Date"])
```

Current date

```
pd.Timestamp.now()
```

pd.Timestamp.now()

Set Index

```
df.set_index("ID")
```

Reset

```
df.reset_index()
```

Value Counts

```
df["City"].value_counts()
```

Unique Values

```
df["City"].unique()
```

Number of Unique Values

```
df["City"].nunique()
```

Correlation

```
df.corr(numeric_only=True)
```

Data Types

```
df.dtypes
```

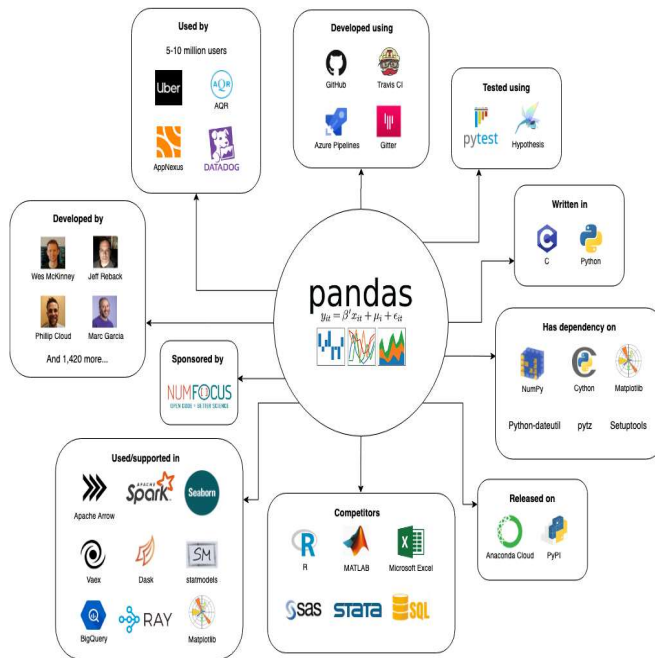
Convert

```
df.astype(float)
```

Advantages of Pandas Easy to use Fast data processing Handles large datasets Supports multiple file formats Powerful filtering and grouping Integrates with NumPy, Matplotlib, and Scikit-learn Ideal for data analysis and machine learning

Disadvantages of Pandas Uses more memory for very large datasets Slower than some specialized big-data tools for extremely large data Not suitable for distributed computing by itself Can become slow with datasets that don't fit into memory

Applications of Pandas Data analysis Data cleaning Machine learning preprocessing Business analytics Financial analysis Scientific research Data visualization (with Matplotlib/Seaborn) Reporting and dashboard preparation



Conclusion

Pandas is one of the most important Python libraries for data analysis and data manipulation. It provides powerful data structures like Series and DataFrame, making it easy to clean, organize, analyze, and transform data. Because of its simplicity, speed, and rich functionality, Pandas is widely used in data science, machine learning, artificial intelligence, finance, business analytics, and scientific research. It is considered an essential tool for anyone working with data in Python.

Double-click (or enter) to edit

Double-click (or enter) to edit

